

Local vs. Global Scope & Scope Resolution

When a local variable shares the same name as a global variable, it "shadows" it. You can pass this shadowing using the Scope Resolution Operator (`::`).

```
>++
```

```
#include <iostream>
using namespace std;

// Global Variable
int x = 40;

int main() {
    // Local Variable (shadows the global 'x')
    int x = 20;

    // A nested local scope
    {
        int x = 50;
        cout << "Most Local x: " << x << endl; // Prints 50
    }

    cout << "Function Local x: " << x << endl; // Prints 20

    // Accessing the Global 'x' using the scope resolution operator '::'
    cout << "Global x: " << ::x << endl;      // Prints 40

    return 0;
}
```

The `const` Keyword with Pointers

Understanding how `const` applies to pointers is a classic interview question. The rule is: **If `const` is before the `*`, the data is constant. If it is after the `*`, the pointer is constant.**

```
>++
```

```
#include <iostream>
using namespace std;

int main() {
    int b = 50;

    // 1. Constant Data, Non-Constant Pointer
    // You cannot change the value at the address, but you can point to a new address.
```

```

const int* ptr1 = new int(10);
// *ptr1 = 20; // ERROR: Data is read-only
ptr1 = &b;      // VALID: Pointer can point somewhere else

// 2. Non-Constant Data, Constant Pointer
// You can change the value, but you cannot make the pointer point anywhere else.
int* const ptr2 = new int(10);
*ptr2 = 20;      // VALID: Data can be modified
// ptr2 = &b; // ERROR: Pointer is locked to this memory address

// 3. Constant Data AND Constant Pointer
// Absolutely locked down. Neither the pointer nor the data can change.
const int* const ptr3 = new int(10);
// *ptr3 = 20; // ERROR
// ptr3 = &b; // ERROR

return 0;
}

```

Initialization Lists

An initialization list is the most efficient way to assign variables when an object is born. It is strictly required for `const` variables because they cannot be assigned *after* creation.

```

}++

#include <iostream>
using namespace std;

class ABC {
public:
    int x;
    int* y;
    const int z; // Must be initialized via an initialization list

    // Modern C++ Initialization List
    ABC(int _x, int _y, int _z) : x(_x), y(new int(_y)), z(_z) {
        // The body can be empty. The variables were initialized the exact
        // moment they were allocated in memory.
        cout << "Values initialized: " << x << ", " << *y << ", " << z << endl;
    }
};

int main() {
    ABC obj(10, 20, 30);
}

```

```
    return 0;
}
```

static Data Members and Methods

static variable is shared across *all* objects of the class. A static method belongs to the class itself, meaning it has no `this` pointer and can only interact with static variables.

C++

```
#include <iostream>
using namespace std;

class ABC {
public:
    // Declaration of a static variable
    static int x;

    // Static member function
    static void printStatic() {
        cout << "Shared Static x is: " << x << endl;
        // cout << this->x; // ERROR: 'this' pointer does not exist here!
    }
};

// Static variables MUST be initialized outside the class!
int ABC::x = 0;

int main() {
    ABC obj1;
    ABC obj2;

    obj1.x = 10;

    // Changing obj1.x changes it for obj2 as well because they share the same memory!
    cout << "obj2 sees x as: " << obj2.x << endl; // Prints 10

    // Static functions are called directly using the Class name
    ABC::printStatic();

    return 0;
}
```

a class dynamically allocates heap memory, the default Shallow Copy will crash your program when destructors run. You must write a Deep Copy constructor.

```
>++

#include <iostream>
using namespace std;

class ABC {
public:
    int x;
    int* y;

    ABC(int _x, int _y) : x(_x), y(new int(_y)) {}

    // CUSTOM DEEP COPY CONSTRUCTOR
    ABC(const ABC& source) {
        x = source.x;
        // Allocate entirely NEW memory on the heap and copy the value over
        y = new int(*(source.y));
    }

    ~ABC() {
        delete y;
    }
};

int main() {
    ABC A(1, 2);

    // Triggers our Custom Deep Copy Constructor
    ABC B = A;

    // Modifying B's heap data does NOT affect A's heap data.
    *(B.y) = 20;

    cout << "A's pointer value: " << *(A.y) << endl; // Prints 2
    cout << "B's pointer value: " << *(B.y) << endl; // Prints 20

    // No Double-Free error will happen here because A and B have separate heap allocations
    return 0;
}
```

Private Constructors and the friend Keyword

If you make a constructor `private`, nobody can create the object directly. You use the `friend` keyword to grant another class exclusive "VIP" access to build the object.

C++

```
#include <iostream>
using namespace std;

class Box {
private:
    int width;

    // Private Constructor: main() cannot call this!
    Box(int w) : width(w) {
        cout << "Box created with width: " << width << endl;
    }

    // Granting BoxFactory exclusive access to all private members
    friend class BoxFactory;
};

class BoxFactory {
public:
    int count = 0;

    // Factory Method to securely create Boxes
    Box getABox(int w) {
        count++;
        // Valid because BoxFactory is a 'friend' of Box
        return Box(w);
    }
};

int main() {
    // Box b(5); // ERROR: Constructor is private!

    BoxFactory factory;
    Box myBox = factory.getABox(10); // SUCCESS: Created via the Factory

    cout << "Total boxes manufactured: " << factory.count << endl;

    return 0;
}
```

Virtual Destructors (Preventing Memory Leaks in Polymorphism)

When you delete a Derived class object through a Base class pointer, C++ will only call the Base destructor. To ensure the Derived destructor is called first (and its memory is freed), the Base destructor **must** be marked `virtual`.

```
> ++
```

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base Constructor" << endl; }

    // CRITICAL: Always make Base destructors virtual when using inheritance!
    virtual ~Base() { cout << "Base Destructor" << endl; }
};

class Derived : public Base {
public:
    int* data;

    Derived() {
        cout << "Derived Constructor" << endl;
        data = new int[1000]; // Allocating heavy memory
    }

    ~Derived() {
        cout << "Derived Destructor (Memory Freed!)" << endl;
        delete[] data;
    }
};

int main() {
    // Upcasting
    Base* b = new Derived();

    // If ~Base() wasn't virtual, ~Derived() would NEVER be called here!
    delete b;

    return 0;
}
```

inline function

In the rhythm of execution, every function call carries a weight—a jump in control, a new frame on the stack, and variables pushed into memory. For heavy logic, this cost is invisible. But for tiny, repetitive tasks, this overhead becomes a bottleneck.

This is exactly where `inline` functions step in.

Here is the deep dive and the complete code implementation for **Inline Functions vs. Macros**, directly addressing the missing piece from the bonus chapter.

The Concept: Bypassing the Stack

When you mark a function as `inline`, you are making a request to the C++ compiler: *"Do not jump to a separate function in memory. Instead, take the code inside this function and paste it directly wherever the function is called."*

- **The Advantage:** It completely eliminates the function-calling overhead (stack creation, memory allocation for parameters, control jumps). Execution becomes blazing fast.
- **The Drawback (Code Bloat):** If you make a massive 200-line function `inline` and call it 50 times, the compiler will paste those 200 lines 50 times into your executable, massively inflating the size of your `.exe` file.
- **Modern Compilers:** Compilers are smart. If you mark a massive function as `inline`, the compiler will simply ignore your request and treat it as a normal function.

Inline vs. Macros (`#define`)

Macros do something similar, but they are handled by the **Preprocessor** before the code is even compiled. A macro blindly does "Find and Replace" on the text.

- **Macros are dangerous** because they lack type-checking and respect no scope boundaries.
- `inline` **is safe** because it is handled by the Compiler, maintaining strict C++ type safety and logical scope.

The Complete Code Implementation

C++

```
#include <iostream>
using namespace std;

// =====
// 1. MACRO (The Preprocessor Way)
// =====
// Blindly replaces text before compilation.
// Fast, but dangerous (no type safety, prone to logical errors).
#define MAX(a, b) ((a) > (b) ? (a) : (b))

// =====
// 2. INLINE FUNCTION (The C++ Way)
// =====
// Handled by the compiler. Type-safe and respects scope.
inline void showNumber(int num) {
    cout << "The number is: " << num << endl;
}

// Another small utility perfect for inlining
inline int square(int x) {
    return x * x;
}
```

```

}

int main() {
    cout << "=== INLINE FUNCTIONS ===" << endl;

    // The compiler sees this: showNumber(10);
    // But internally, it pastes the actual code: cout << "The number is: " << 10 <<
    showNumber(10);
    showNumber(20);

    int sq = square(5); // Replaced with: int sq = 5 * 5;
    cout << "Square of 5 is: " << sq << endl;

    cout << "\n=== MACROS ===" << endl;

    // The preprocessor blindly replaces MAX(100, 200) with:
    // ((100) > (200) ? (100) : (200))
    int highest = MAX(100, 200);
    cout << "Highest value is: " << highest << endl;

    return 0;
}

```

The Takeaway for DSA & Production

When writing algorithms or high-performance systems, milliseconds matter. Keep your heavy logic structured, but for those tiny 1-to-3 line helper functions (like returning a `max` value or updating a single pointer), mark them `inline`. It keeps your source code clean and readable without sacrificing the raw, unyielding speed of C++.

